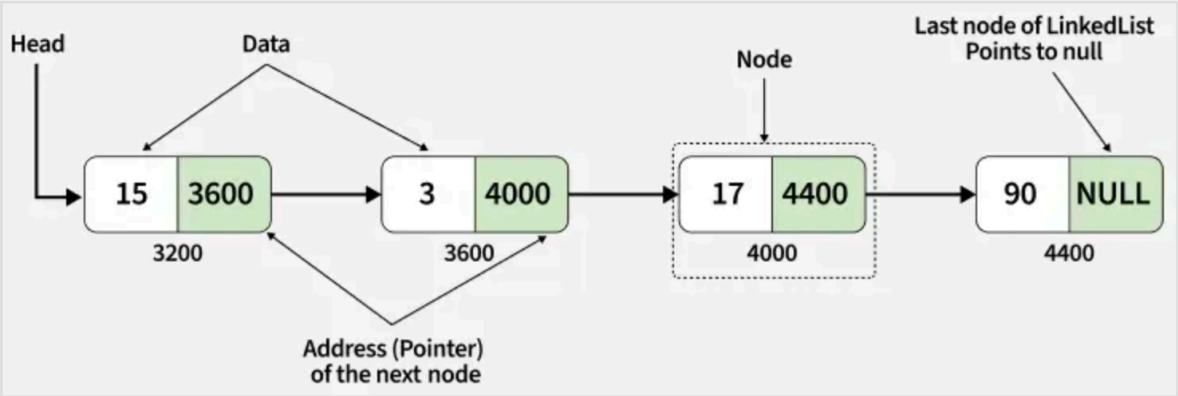
Day 4

Linked list:

It is a type of linear data structure where individual items are not necessarily at contiguous locations. The individual items are called "**nodes**" and connected with each other using "**links**".
A node contains two things -

- Data
- Link that connects it with another node (address)

The **first node** is called the "***head node***" and the whole list can be traversed using this head and next links.



Singly linked list:

It is a fundamental data structure that consists of **nodes** where each node contains a **data field** and a **reference** to the next node in the linked list. The next of the last node is **Null**, indicating the end of the list.

- It supports efficient insertion and deletion operations.
  - Each node consists of data and a pointer to the next node, which allows nodes to be dynamically linked together like a chain sequence.




```python
# Definition of a Node in a singly linked list
class Node:
    def __init__(self, data):

        # Data part of the node
        self.data = data
        self.next = None
```

```python
# constructor to initialize a new node with data
class Node:
    def __init__(self, new_data):
        self.data = new_data
        self.next = None

# Create the first node (head of the list)
head = Node(10)

# Link the second node
head.next = Node(20)

# Link the third node
head.next.next = Node(30)

# Link the fourth node
head.next.next.next = Node(40)

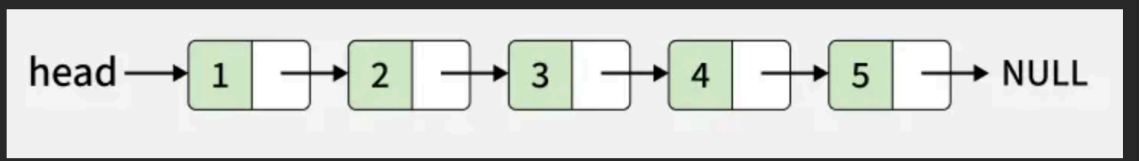
# printing linked list
temp = head
while temp is not None:
    print(temp.data, end=" ")
    temp = temp.next
```

Output

```
10 20 30 40
```

Traversal of singly linked list:
Fundamental operation where we traverse/visit each node of the linked list.

Input:



Output: 1 -> 2 -> 3 -> 4 -> 5

```python
# a linked list node
class Node:
    def __init__(self, new_data):
        self.data = new_data
        self.next = None

# function to traverse and print the singly linked list
def traverseList(head):
    while head is not None:
        print(head.data, end="")
        if head.next is not None:
            print(" -> ", end="")
        head = head.next
    print()

if __name__ == "__main__":

    # create a hard-coded linked list:
    # 10 -> 20 -> 30 -> 40
    head = Node(10)
    head.next = Node(20)
    head.next.next = Node(30)
    head.next.next.next = Node(40)

    traverseList(head)
```

Output

```
10 -> 20 -> 30 -> 40
```

Insertion in singly linked list:
Inserting a new node at the front -

```python
# Define a node in the linked list
class Node:
    def __init__(self, x):
        self.data = x
        self.next = None

# Function to insert a new node
# at the beginning of the list
def insertAtFront(head, x):
    newNode = Node(x)
    newNode.next = head
    return newNode

# Function to print the contents
# of the linked list
def printList(head):
    curr = head
    while curr is not None:
        print(curr.data, end="")
        if curr.next is not None:
            print(" -> ", end="")
        curr = curr.next
    print()

if __name__ == "__main__":
    # Create the linked list 2->3->4->5
    head = Node(2)
    head.next = Node(3)
    head.next.next = Node(4)
    head.next.next.next = Node(5)

    # Insert a new node at
    # the front of the list
    x = 1
    head = insertAtFront(head, x)

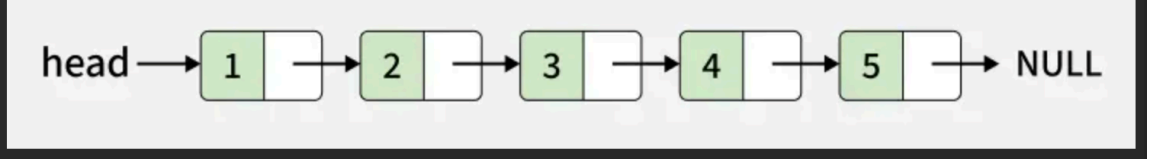
    # Print the updated list
    printList(head)
```

Output

```
1 -> 2 -> 3 -> 4 -> 5
```

Inserting a node at the end -

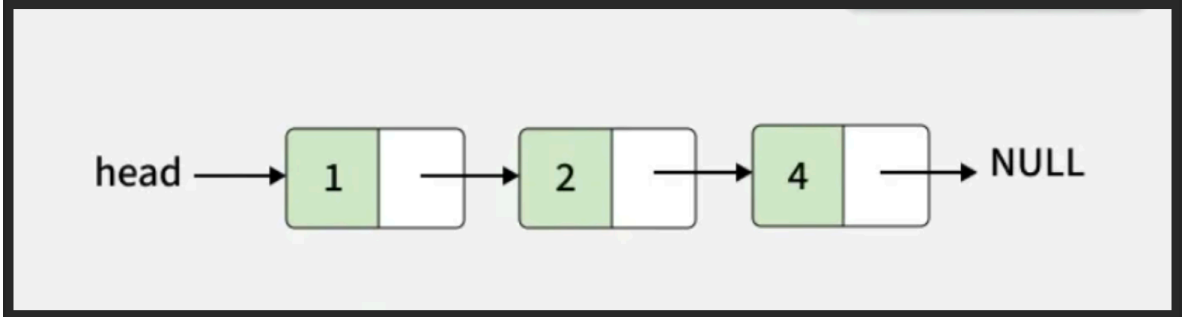
Input: *x = 6*



Output: *1 -> 2 -> 3 -> 4 -> 5 -> 6*

Inserting a node at a specific position -

***Input**: val = 3, pos = 3*

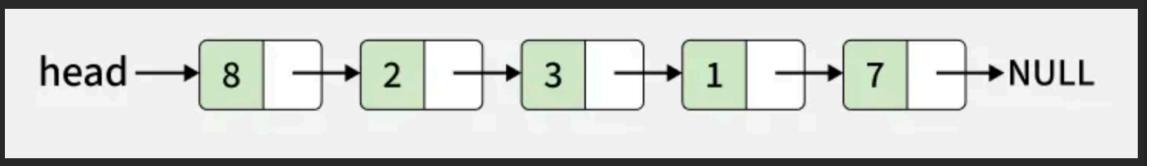


***Output**: 1 -> 2 -> 3 -> 4*

Deletion in a singly linked list:

Deletion at beginning -

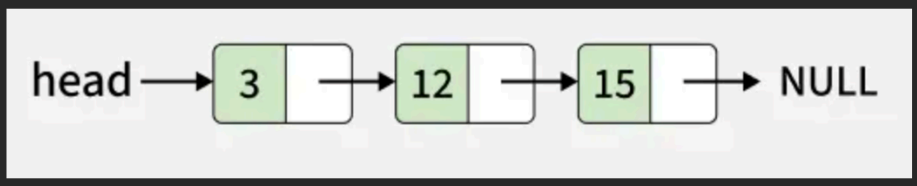
Input :



***Output :** 2 -> 3 -> 1 -> 7 -> NULL*

Deletion at end -

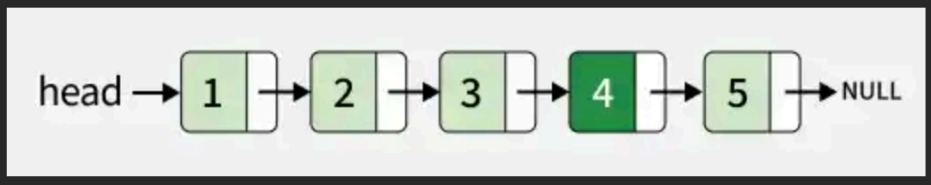
Input:



***Output:** 3 -> 12 -> NULL*

Deletion at a specific position -

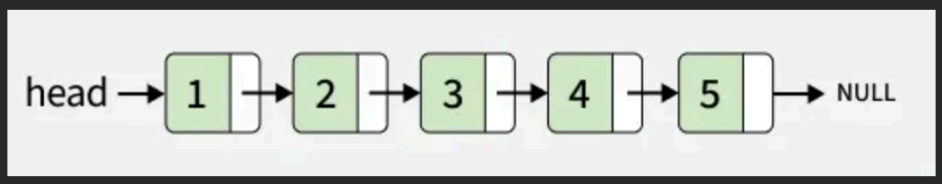
***Input:** x = 4,*



***Output:** 1 -> 2 -> 3 -> 5*

Reversing a linked list:

Input:



***Output**: 5 -> 4 -> 3 -> 2 -> 1*

Two pointer technique:
It is a common technique used to solve problems which involves two pointers that traverse the linked list at different speeds/directions.
Use-cases -

1. Finding the middle of a linked list
2. Reversing a linked list
3. Finding the nth node from the end of the linked list
4. Finding the insertion point of two linked lists.

Creating a linked list:

```
#creating a linked list

class Node:
  def __init__(self, data):
    self.data = data
    self.next = None

node1 = Node(10)
node2 = Node(20)
node3 = Node(30)
node4 = Node(40)

node1.next = node2
node2.next = node3
node3.next = node4

current = node1
while current is not None:
  print(current.data, end="->")
  current = current.next
print("None")
```

```
10->20->30->40->None
```

Inserting a node:

```python
#inserting a node

class Node:
  def __init__(self, data):
    self.data = data
    self.next = None

node1 = Node(10)
node2 = Node(20)
node3 = Node(30)
node4 = Node(40)

node1.next = node2
node2.next = node3
node3.next = node4

new_node = Node(50)

#inserting new node after 2nd node
current = node1
while current is not None and current.data != 30:
  current = current.next
new_node.next = current.next
current.next = new_node

#printing all nodes
current = node1
while current is not None:
  print(current.data, end="->")
  current = current.next
print("None")
```

```
10->20->30->50->40->None
```

Deleting a node:

```
#deleting a node

class Node:
  def __init__(self, data):
    self.data = data
    self.next = None

node1 = Node(10)
node2 = Node(20)
node3 = Node(30)
node4 = Node(40)

node1.next = node2
node2.next = node3
node3.next = node4

#deleting the node from the beginning
if node1 is not None:
  node1 = node1.next      #updating head node to next node

current = node1
while current is not None:
  print(current.data, end="->")
  current = current.next
print("None")
```

```
20->30->40->None
```

Reverse a linked list:

```python
#reverse a linked list

class Node:
  def __init__(self, data):
    self.data = data
    self.next = None

node1 = Node(10)
node2 = Node(20)
node3 = Node(30)
node4 = Node(40)

node1.next = node2
node2.next = node3
node3.next = node4

#reversing
prev = None
current = node1
while current is not None:
  next_node = current.next
  current.next = prev     #flipping the pointer
  prev = current          #moving prev forward
  current = next_node     #moving current forward
head = prev       #prev is the new head

current = head
while current is not None:
  print(current.data, end="->")
  current = current.next
print("None")
```

```
40->30->20->10->None
```

Finding the middle node:

```python
#finding the middle node

class Node:
  def __init__(self, data):
    self.data = data
    self.next = None

node1 = Node(10)
node2 = Node(20)
node3 = Node(30)
node4 = Node(40)
node5 = Node(50)

node1.next = node2
node2.next = node3
node3.next = node4
node4.next = node5

slow = node1
fast = node1
while fast is not None and fast.next is not None:
  slow = slow.next
  fast = fast.next.next
print("Middle node: ", slow.data)
```

```
Middle node:  30
```